

MicroKernel Architecture Practice

AVEPOINT, INC.

December 12, 2010

Authored by: Baron

MicroKernel 架构实践

<i>Revision</i>	<i>Author</i>	<i>Description</i>
1 (12/12/2010)	Baron	Initial draft
2 (03/2 /2011)	Baron	Add Remoting type key to invoke specific implement instead of default implement of the interface
3(05/24/2011)	Baron	As the code of the microkernel restructured, the dependency and Host process of the MicroKernel is changed, so we changed the documentation of MicroKernel architecture.
4(06/7/2011)	Baron	Add the MicroKernel internal endpoint cache policy, this policy keep the underlying connection alive.

Use, duplication, or disclosure by the Government is subject to restrictions as set forth in subdivision (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 (Oct. 1988) and FAR 52.227-19 (C) (June 1987).

Copyright © AvePoint, Inc. 2010

Unpublished - All rights reserved under the copyright laws of the United States.

Contents

1.	微核模型的引入.....	- 3 -
1.1.	微核编程模型的背景.....	- 3 -
1.2	微核解决方案.....	- 4 -
2.	微核的一般理论.....	- 5 -
2.1.	微核设计目标.....	- 5 -
2.2.	微核整体设想以及架构:.....	- 5 -
2.3.	隐藏微核实现.....	- 6 -
2.4.	设计微核.....	- 6 -
3.	微核实践.....	- 8 -
3.1.	微核编程模型.....	- 8 -
3.2	DocAve 中微核依赖关系.....	- 8 -
3.2.1	微核与 .Net Framework 以及 Visual studio 的关系.....	- 9 -
3.2.2	微核的显式依赖关系.....	- 9 -
3.2.3	微核的隐式依赖关系.....	- 11 -
3.3.	DocAve 中微核实现说明.....	- 12 -
3.3.1.	微核定义.....	- 13 -
3.3.2.	微核功能意图定义.....	- 16 -
3.3.3.	实现外部服务器.....	- 18 -
3.3.4.	适配器/代理以及核心内部服务器.....	- 20 -
3.3.5	微核的附加功能.....	- 23 -
4	微核应用.....	- 25 -
4.1	微核的承载.....	- 25 -
4.1.1	DocAve6 Manager 中的微核承载.....	- 25 -
4.1.2	DocAve6 Agent, Media, Report 服务中的微核承载.....	- 25 -
4.2	使用微核.....	- 26 -
5	关于微核的最后思考.....	- 30 -

1. 微核模型的引入

一般说来，一种新技术或新方法之所以出现，是因为现实中遇到了无法解决的问题，同时新技术或新方法也在一定程度上解决了问题。与之类似，微核思想引入 DocAve 编程模型，也是为了解决 DocAve 开发实践中遇到的问题。微核的本质出发点只有一条：在不引入额外复杂性的前提下，对大规模 SOA 应用服务程序进行有效管理。

1.1. 微核编程模型的背景

正如我们所知道的，DocAve6 通讯模型底层采用了 WCF 框架。根据微软的官方定义：WCF (Windows Communication Foundation) 是用于构建面向服务的应用程序的框架。借助 WCF，可以将数据作为异步消息从一个服务终结点发送至另一个服务终结点。服务终结点可以由 IIS 承载的持续可用的服务的一部分，也可以是应用程序中承载的服务。终结点可以从服务终结点请求数据的服务客户端。简单消息可以是作为 XML 发送的单个字符或单个单词，复杂消息可以是二进制数据流。同时 WCF 框架通过精心的设计，为我们提供了基于异构网络的方便灵活的通讯基础设施，带来前所未有编程体验。这种方便灵活性，从某种意义上讲，就是层次性，复杂性，这正验证了那句至理名言：表面平等的背后掩盖着等级。当我们真正融入其中的时候，WCF 框架本身复杂性就充分的展示了出来，同时也对我们现在的开发带来了严重挑战：

DocAve6 作为一个集成平台，拥有数量众多的模块，按照最初的设计，这些模块消费了数量更多的应用服务，且这些服务之间的关联不大。从一个完整的 WCF 服务的使用来看，每一个应用服务都至少需要一个 WCF 终结点来处理，这其中包括 WCF 服务的配置，终结点的配置，客户端的配置，绑定的配置，以及结合 IOC 框架的配置等。任何一处配置错误，服务都不能工作，甚至，整个的 DocAve6 集成平台都不能运行，这绝不是危言耸听，WCF 配置错误导致 DocAve6 系统没有办法运行的问题已经切切实实的存在了。并且这种问题足够细微，非编译时错误，问题不易定位等。

随着 DocAve6 开发规模的越来越大，这种工作量一直增大并且会一直增大下去，最重要的是，每一个新进入 DocAve6 的开发人员都面对同样的学习成本曲线，Wcf 框架的学习，IOC 框架的学习，维护愈来愈大的配置文件等。

与 DocAve6 平台愈来愈多的 WCF 服务相伴随的另外一个更为严重的问题是：DocAve6 平台暴露了太多的服务与终结点。按照当前 DocAve6 系统的设计，Manager 服务是无状态并且是任意可访问的。从安全的角度来说，DocAve6 安全设计还很脆弱：借助于相关代理生成工具，我们很容易得到 DocAve 系统太多的细节，并且可以轻易的使用自己构建的 Wcf 代理对底层服务发起调用，而这种调用是不受任何控制的。这就好比一个脆弱的交通系统，到处都是可以通行的路口，即使有红绿灯，也是不可能有效降低交通事故发生的概率的，也是无法把混乱脆弱的交通重整为一个健康安全的交通系统的。一般说来，任何一个终结点的公开，都应该经过严格定义与安全评估，因为只有这样，才能使整个系统的安全有所保障，同样也只有有效的管理 DocAve6 公开的终结点的数量，才能谈得上对整个 DocAve6 系统的安全性进一步增强。一言以蔽之，尽可能的减少

DocAve6 系统的攻击面。

鉴于目前的这种分散编程状态以及由其带来的安全方面的问题，我们很有必要对这种混乱的状态进行统一的管理（注意这里我们使用管理一词而不是控制一词，是因为我们认为整个问题的解决过程少了一些蛮横多了一些艺术的意味，体现了更高层次的抽象与思考）。这个问题的解决方案就是微核。

1.2 微核解决方案

微核的概念已经存在很久了，微核的概念通常是和操作系统联系在一起的。例如 Windows NT 的早期版本中就有微核。微核都是通过提供一种核心基础结构来完成特定的目的。

在 DocAve6 中实现的微核，通过对 WCF 服务概念进行更高层次的抽象：一切应用服务都是微核的外部服务器，来使应用服务不再直接依赖于 WCF 框架而是依赖于微核框架。

应用服务从对 WCF 框架的依赖转而依赖微核框架，这在客观上就造成了一种可能性：微核有可能对应用服务进行行之有效的管理，另外，微核通过转移依赖关系，使整个开发组织学习与维护程序的成本也大幅降低。这点可以从以下例子得到切实的说明：如关于 WCF 知识的掌握，很大程度上由以前的各个开发人员转移给了微核的实现者，又如通过微核依赖于 WCF 这种结构，所有关于服务，绑定，配置的工作也基本上由微核的实现者来完成。

理论上讲，也更为重要的是：整个系统只有关于微核的服务信息，配置，以及为了适应微核而保留的一些基础配置。这一点，带来了以下可能性：

- ✧ 有效的控制了服务规模的扩张。微核通过公开服务方式的转变，大幅的降低了配置文件的数量，在降低编程难度提高编程体验的同时，对服务进行了行之有效的管理，因为公开的只是关于微核的配置。
- ✧ 增强的安全性。微核通过对应用程序编程接口（API）的控制，把调用信息封装进了适配器代理。对微核而言，只有适配器才知道如何有效的调用微核，微核也只理解适配器的调用。其他的针对微核构建的代理都很难把对微核的调用转化为有效的调用，因为调用细节是隐藏的，且调用需要复杂的底层细节和技术。

微核虽然在客观上提供多种优点，但毕竟只是提供了一种可能性，要使得这种抽象的可能性转化为现实性，需要在进一步检验与改进微核的性能和稳定性的同时，不断增强微核功能。

总之，**微核是一种对分布式服务管理的抽象**。通过在 DocAve6 中实现微核架构，在大幅降低开发难度，增强系统安全性的同时，也增加了 DocAve6 系统的弹性与可伸缩性。**微核通过实现模式来提供弹性，依托终结点来实现可伸缩**。除此之外，从微核代码的角度来说，微核还被定义为一个除了核心服务之外的完全可替代的系统，在提供弹性与可伸缩性的同时，整个微核也实现了**可测试性**（易于编写运行单元测试）。

2. 微核的一般理论

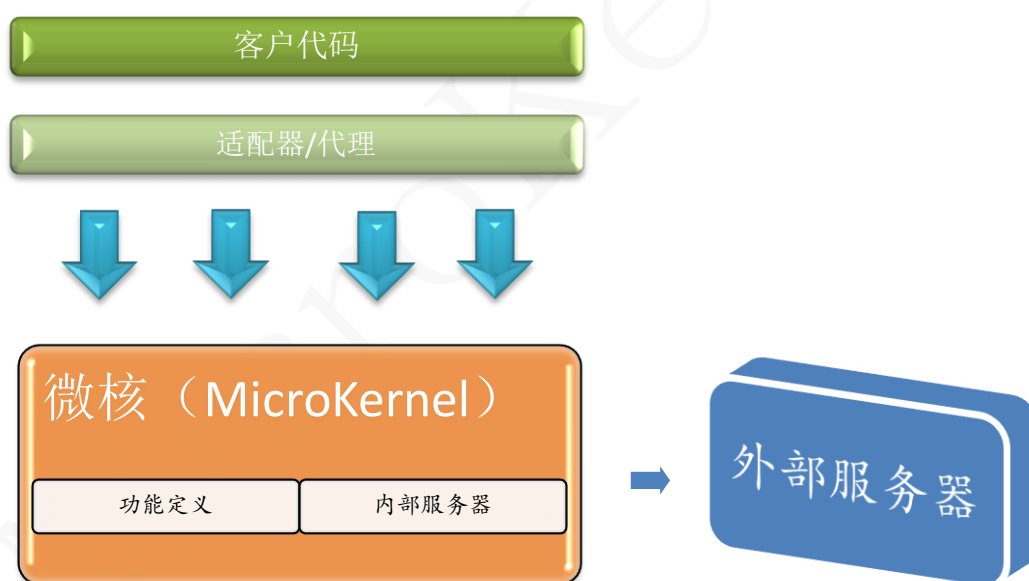
在本节，我们将对微核的基本理论进行较为详细的介绍，从理论上说明微核设计的优越特征。

2.1. 微核设计目标

微核的目标是使应用程序能够应对未来的考验，以便应用程序在变化时不必对整个应用程序进行大量的重新编码，进一步说，微核的核心目标保持核心应用稳定的同时提供足够的弹性。

2.2. 微核整体设想以及架构：

从一般意义上将，微核的规则相当简单，核心架构图如下所示：



上图中，定义了四个功能模块：客户代码，适配器，微核，以及外部服务器，在微核中还列明了两个方面，功能定义以及内部服务器。

客户代码是微核的使用者，客户代码只能通过适配器或者代理提供的功能与微核交互，客户代码并不知道微核的内部工作细节甚至无需知道微核的存在。

微核定义了应用程序的基本逻辑，并且实现了应用程序的基本逻辑。定义了应用程序的基本逻辑意味着定义了接口以及抽象基类等。实现应用程序的基本逻辑意味着定义了如何定位加载程序集，如何加载配置文件等。内部服务器是微核的 `gopher` 类，对基本的功能提供定位，加载，

导航服务。内部服务器用来执行业务逻辑实现的功能块，但从微核架构的角度说，他并不是微核的一部分，因为内部服务器并不直接和微核相关，将内部服务器加载到微核将使微核膨胀。外部服务器是客户代码间接使用的功能块，微核作为中间实体提供服务。

简单的说，**微核思想的本质是创建一种能够支持多个应用程序版本的架构，变化的是单个模块的实现，但微核的核心约定保持不变。**

2.3. 隐藏微核实现

我们对微核进行自上而下的分析，客户代码与适配器/代理交互，而客户代码并不知道微核的实现，有意这样做的目的是，微核，内部服务器，外部服务器实现了一些功能，这些功能将使用诸如拦截器，事件模型，序列化，反射等技术以及明了类型之间的关系的规则。为了使微核发挥作用，需要在微核层面组装功能，这需要微核知悉或者定义一些潜在的规则，但客户不需要知悉。

在整个的结构中，适配器或者代理的目的之一，就是作为客户代码和微核的桥梁，因为他知道子系统是做什么的，并且知道怎么做。

总而言之，适配器完成以下主要功能：

- ◇ 隐藏微核的复杂性；
- ◇ 隐藏微核的依赖性；
- ◇ 为客户代码，外部服务器和微核之间提供桥梁。

适配器设计的足够聪明，因为适配器本身知道或者由它缔造了一个规则：将所有的外部服务器拼凑起来，并知道如何调用微核。实际上，适配器就是微核的应用程序编程接口（API），客户只需要了解该 API。

2.4. 设计微核

微核思想的实现足够复杂，因为在需要明确界定微核逻辑概念的同时，实现上可能是一个内部服务器或者外部服务器甚至是适配器或者代理的一部分。因此理解微核的最好方式是了解微核模块的任务。

一般说来，微核需要完成以下核心的任务：

- ◇ 提供内部服务器，外部服务器以及相关资源的访问；
- ◇ 提供绑定内部服务器，外部服务器的框架；
- ◇ 以抽象的形式提供核心的功能，也就是尽量少的创建真实的实现，提供的是所有被其他模块使用的定义。

在设计微核时，主要目标是保持微核的**紧凑性以及聚焦性**，微核应该提供基本功能并且委托其他事情，微核应尽量避免更新，从应用程序的角度来说，微核总是使用接口和抽象基类。

在微核的设计中，Bridge 模式（桥接）与微核紧密结合，因为适配器代码使用的是外部服

务器使用的逻辑。

内部服务器完成下面的核心任务：

- ✧ 实现微核之外的附加逻辑；
- ✧ 封装与微核之间的系统细节。

内部服务器有一个核心的任务，是微核能够正常的运行，理想情况下，内部服务器会提供一个中性的接口，以便能够定位配置与外部服务。

微核也是最丑陋的代码，因为他总是试图在抽象与过度复杂之间折中，因此丑陋在所难免。

外部服务器必须完成下面的任务：

- ✧ 提供微核与适配器或者代理使用的编程接口。

外部服务器的定义也很简单，因为他们只需要完成微核，客户代码，适配器代理期望他完成的功能。

3. 微核实践

本节，我们将就 DocAve6 中实现的微核以及微核理论的实践进行详细的介绍，主要包括微核使用的设计模式，微核依赖的组件，以及微核的具体等。

3.1. 微核编程模型

在下一部分，我们将就微核在 DocAve6 中的实现以及对微核理论遵守进行详细说明。在开始之前，我们就 DocAve6 中微核使用的编程模式做一简要说明。

DocAve 微核的实现依赖于两大编程模式：

➤ Client-Dispatcher-Server 模式

此模式的正式定义中定义了客户代码，服务器和分配器，客户代码要求分配器提供一个服务器的引用，然后使用所提供的服务，服务器在分配器中自己注册自己并为客户代码提供服务，分配器负责从客户代码到服务器的交叉引用服务请求，发现服务器后，分配器负责实例化服务。在当前的实现中，我们采用的是静态的分配器，静态分配器的意思是说，一旦我们加载了程序集，我们便不能对服务进行动态的卸载。静态分配器非常类似于插件架构，但提供了有别于插件的附加特征，其中一个附加的特征就是分配器必须有将标识符解析成程序集以及其中类型的能力。

作为通用的规则，分配器一般通过以下的方式进行程序集以及类型解析：

- ✧ 配置文件；
- ✧ 程序集目录；
- ✧ 工厂委托；
- ✧ 服务器注册；

当前的微核实现中，基本上采用了配置文件加工厂委托的方式。关于此点，微核的实现说明中会有详细的说明。

➤ Pipes and Filter 模式

从一般意义上讲，微核通过此模式的使用，把整个的消息处理流程抽象为一个顺序的工作流，因此，从理论上讲，微核的整个核心流程部分是可以 WF 抽象并且实现的，当前实现中是把流程抽象为一系列的消息过滤器，并进行了外层委托。

3.2 DocAve 中微核依赖关系

在 DocAve 中实现的微核，显式或者隐式的依赖一些因素，这些依赖因素成为在微核设计与实现时的出发点与采用方式的基本依据。

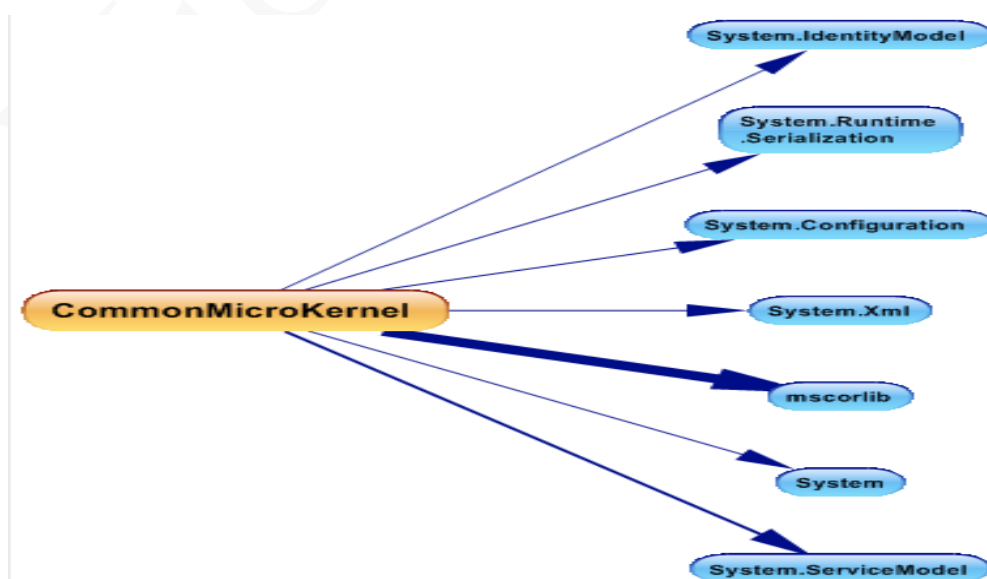
3.2.1 微核与 .Net Framework 以及 Visual studio 的关系

在 DocAve6 中，为了兼容不同的使用场景，微核基于 .net framework 3.0 实现，至于具体的原因，我们相信大家都明了一个不是理由的理由，附加的 .net 版本需要额外的资源，我们是在为客户思考，而不是在为开发思考。从一般的开发使用者的角度来说，.net 版本之间的区别对开发可以说是透明的，因为我们一般不针对某个 .net 提供的特殊技术进行编码。不幸的是，这仅仅是我们的一个美好愿景，在 DocAve 中的 GCommon 组件，被强制定义为基于版本 3.0 的 .net framework 进行开发，而 Server 端程序，都基于更高版本的 .net framework 进行开发。一般说来，即使不同的版本，我们也有相互调用的可能，这个结论来源于以下的事实：在 .net 4.0（不包含 4.0 版本）以下，2.0 以上版本的 framework 中，采用的是同一个公共语言运行时 CLR2.0，也就是说，基于以上 .net 版本范围开发的程序，相互调用从理论上应该是没有问题的，甚至有可能编译时进行引用也没有问题。不过这种美好的愿景在 DocAve6 的当前开发中并没有成为现实。这个论断基于以下两个无法改变的事实，DocAve 整个开发基于 visual studio 2010，visual studio 2010 增强了对编译时引用的限制。

尽管存在实质上行得通的调用关系，但是在 visual studio 2010 中，其代码编辑器子组件已经被严格的定义成不能引用高于当前工程的 .net 组件，这就造成了一个问题，我们需要在微核绕过这种编译时的限制，来拓展微核使用的 .net framework 版本的限制。微核通过明了一个概念，来实现了一个完成这种功能的子系统。这个概念被称作运行时类型发现，实现的子系统被称为服务定位器，虽然从实现代码角度来看并不多，但是其却使之前的愿景成为现实：复杂的跨 .net framework 版本的调用成为现实并且成功绕过了 visual studio 2010 强制限制规则。

3.2.2 微核的显式依赖关系

微核的开发，在编译时显式的依赖于七个组件，如下图所示，该图由 NDepend 代码分析软件生成，显式的指明了在编译时微核依赖的组件：

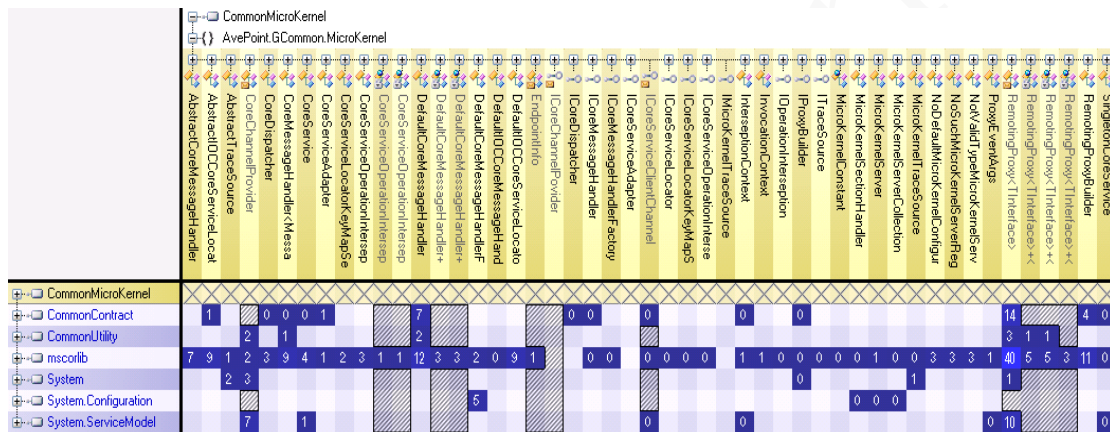


值得一提的是，图中的线条的粗细指明了微核对另外一个组件依赖程度的强弱关系，当前图使用的依赖关系指标是 member，该指标由 NDepend 定义。可以看出，对 mscorlib 程序集中 member 依赖关系是最强的。

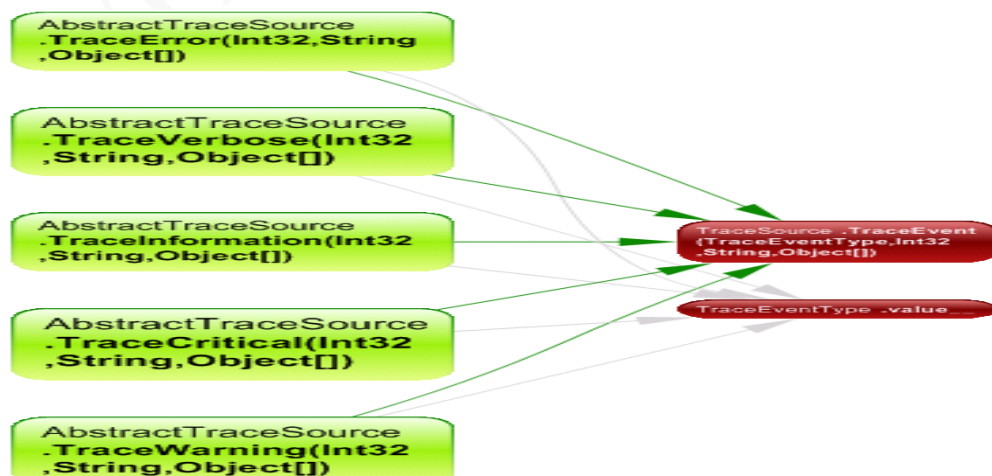
下面一一说明相关组件的作用：

- ✧ mscorlib 与 System 程序集，提供 .net 基础类型系统。
- ✧ System.ServiceModel 程序集，提供微核内置核心 Wcf 服务的相关类型。
- ✧ System.IdentityModel 程序集，提供微核底层使用的身份验证相关类型。
- ✧ System.Xml 与 System.Runtime.Serialization 程序集，提供 WCF 序列化功能相关类型。
- ✧ System.Configuration 程序集，提供完成微核内部的外部服务器发现模型的相关类型。

为了对这些依赖关系进行进一步的明确，我们用了另外一幅图来展示当前微核实现中的真实依赖关系，如图所示：



此图的横标列出了微核组件中的所有类型，纵标列出了对相应的组件的依赖次数，在实际的操作中，可以对相关的关系进行进一步的展开，以明确更具体的依赖关系，比如我们查看 AbstractTraceSource 类对 System 程序集的依赖关系，如下图所示：



根据这些复杂的 report，我们也实现在代码编写过程中对微核模块进行及时的重构，不断优化微核结构，使微核符合一些业界定义的一些编码要求（可维护指数，圈复杂度，继承深度指数等）。

微核组件的整体分析结果如下图所示：

Hierarchy	Maintainability Index	Cyclomatic Compl...	Depth of Inheritance	Class Coupling	Lines of Code
GCommon.MicroKernel (Debug)	89	389	3	126	572

下图是进一步进行细粒度分析的结果（可缩放查看），从得分上看微核程序集基本上遵循了良好的编码风格。

Hierarchy	Maintainability Index	Cyclomatic Compl...	Depth of Inheritance	Class Coupling	Lines of Code
GCommon.MicroKernel (Debug)	89	389	3	126	572
AvePoint.GCommon.Contract	92	12	1	0	13
AvePoint.GCommon.MicroKernel	89	377	3	126	559
AbstractCoreMessageHandlerFactory	76	12	1	7	23
AbstractIOCCoreServiceLocator	77	18	1	11	31
AbstractTraceSource	90	18	1	3	16
ClientChannelManager	65	27	1	21	61
ClientChannelWrapper	93	7	1	3	9
CoreChannelIdentityVerifier	88	3	2	5	6
CoreChannelProvider	71	3	1	13	8
CoreDispatcher	90	6	1	8	10
CoreMessage	94	5	1	3	5
CoreMessageHandler<TMessage>	82	15	1	11	23
CoreService	82	11	1	12	19
CoreServiceAdapter	92	4	1	4	5
CoreServiceInvocationContext	93	21	1	3	21
CoreServiceLocatorKeyMapService	84	6	1	3	8
CoreServiceOperationInterception	83	9	1	6	13
DefaultCoreMessageHandler	66	21	2	14	47
DefaultCoreMessageHandlerFactory	70	8	2	13	21
DefaultIOCCoreMessageHandlerFactory	91	4	2	5	6
DefaultIOCCoreServiceLocator	74	13	2	10	9
EndpointInfo	90	16	1	2	19
IChannelProvider	100	1	0	1	0
IDispatcher	100	1	0	1	0
IMessageHandler	100	2	0	2	0
IMessageHandlerFactory	100	1	0	1	0
IService	100	2	0	3	0
IServiceAdapter	100	1	0	1	0
IServiceClientChannel	100	0	0	7	0
IServiceLocator	100	2	0	0	0
IServiceLocatorKeyMapService	100	1	0	0	0
IServiceOperationInterception	100	2	0	1	0
IMicroKernelTraceSource	100	0	0	1	0
InterceptionContext	94	5	1	2	5
InvocationContext	94	7	1	3	7
IOperationInterception	100	2	0	1	0
IProxyBuilder	100	7	0	5	0
ITraceSource	100	15	0	0	0
MicroKernelConstant	63	1	1	0	10
MicroKernelInternalInvocationException	94	4	2	3	4
MicroKernelSectionHandler	93	2	3	5	3
MicroKernelServer	91	13	2	4	13
MicroKernelServerCollection	92	4	3	4	6
MicroKernelTraceSource	94	3	2	6	3
NoDefaultMicroKernelConfigurationSectionRegisteredException	94	4	2	3	4
NoSuchMicroKernelServerRegisteredException	94	4	2	3	4
NotValidTypeMicroKernelServerException	94	4	2	3	4
ProxyEventArgs	94	5	2	3	5
RemotingProxy<TInterface>	68	35	2	34	76
RemotingProxyBuilder	73	12	1	13	33
SerializerHelper	69	10	1	10	22

微核的显式依赖虽然能保证微核在编译时成功被编译,但却不能使微核在运行时成功的运行,微核的成功运行依赖于一些隐式条件。

3.2.3 微核的隐式依赖关系

在 DocAve6 中实现的微核，为了能够正常运行并且发挥功效，需要依赖于一个 IOC 或者说 DI 容器，换句话说，微核本身就被实现为容器的组件，默认情况下，IOC 使用的 Castle Windsor 容器，主要由以下四个程序集组成：

✧ Castle.Core

- ✧ Castle.MicroKernel
- ✧ Castle.Windsor
- ✧ Castle.DynamicProxy2

为了驱动微核，还需要一个能整合容器和微核的中介，这个中介就是微核的配置文件：CommonMicroKernelIocConfigurations.config，此配置文件本质上是一个 castle windsor 容器的配置文件，它定义了为了驱动微核需要的相互依赖关系，能够使微核在运行时正确的进行类型注入。定义如下：

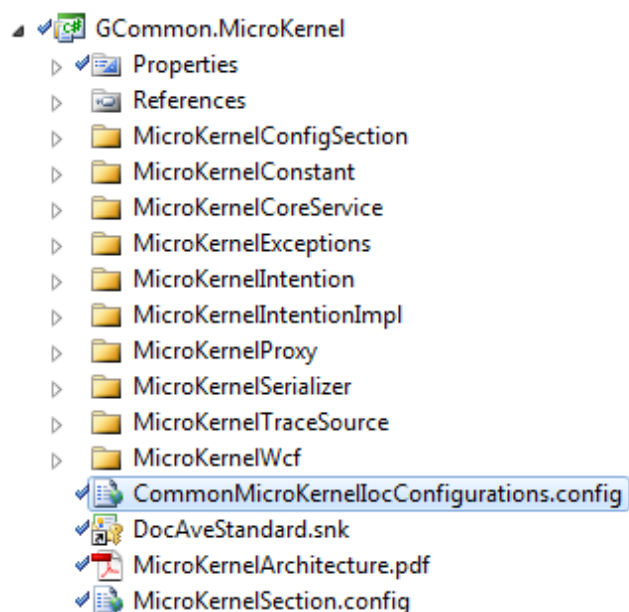
```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <components>
    <component id="AvePoint.GCommon.MicroKernel.ICoreService"
service="AvePoint.GCommon.MicroKernel.ICoreService, CommonMicroKernel"
type="AvePoint.GCommon.MicroKernel.CoreService, CommonMicroKernel" />
    <component id="AvePoint.GCommon.MicroKernel.ICoreDispatcher"
service="AvePoint.GCommon.MicroKernel.ICoreDispatcher, CommonMicroKernel"
type="AvePoint.GCommon.MicroKernel.CoreDispatcher, CommonMicroKernel" />
    <component id="AvePoint.GCommon.MicroKernel.ICoreMessageHandlerFactory"
service="AvePoint.GCommon.MicroKernel.ICoreMessageHandlerFactory, CommonMicroKernel"
type="AvePoint.GCommon.MicroKernel.DefaultIOCCoreMessageHandlerFactory, CommonMicroKernel" />
    <component id="AvePoint.GCommon.MicroKernel.CoreMessage"
service="AvePoint.GCommon.MicroKernel.ICoreMessageHandler, CommonMicroKernel"
type="AvePoint.GCommon.MicroKernel.DefaultCoreMessageHandler, CommonMicroKernel" />
    <component id="AvePoint.GCommon.MicroKernel.ICoreServiceLocator"
service="AvePoint.GCommon.MicroKernel.ICoreServiceLocator, CommonMicroKernel"
type="AvePoint.GCommon.MicroKernel.DefaultIOCCoreServiceLocator, CommonMicroKernel" />
    <component id="AvePoint.GCommon.MicroKernel.IProxyBuilder"
service="AvePoint.GCommon.MicroKernel.IProxyBuilder, CommonMicroKernel"
type="AvePoint.GCommon.MicroKernel.RemotingProxyBuilder, CommonMicroKernel" />
    <component id="AvePoint.GCommon.MicroKernel.IMicroKernelTraceSource"
service="AvePoint.GCommon.MicroKernel.IMicroKernelTraceSource, CommonMicroKernel"
type="AvePoint.GCommon.MicroKernel.MicroKernelTraceSource, CommonMicroKernel" />
  </components>
</configuration>
```

配置文件中定义的类型将在接下来的微核实现说明中进行详细的分析。

3.3. DocAve 中微核实现说明

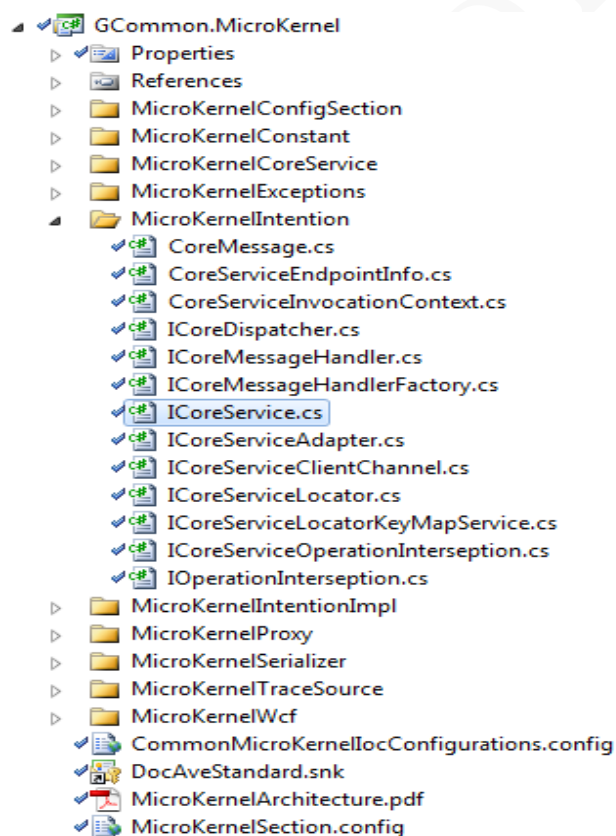
在 DocAve 中，微核被实现为一个 .net 的程序集组件，其 SVN 地址为 <https://10.1.0.130/DocAve/trunk/common/GlobalCommon/GCommon.MicroKernel/>

工程结构如下：



3.3.1. 微核定义

在微核定义中，已经指明了微核的作用是定义并实现应用程序逻辑。根据 DocAve6 系统的特点，整个应用程序的逻辑被抽象为一个消息处理系统。循此逻辑，我们的本质意图是定义一个消息处理系统，用术语描述这个过程就是 **Intention Definition**，此定义位于 MicroKernelIntention 文件夹下工程下，如下图：



该接口的定义如下：

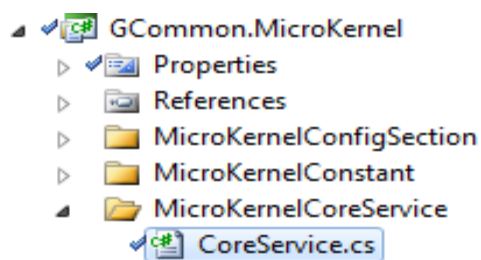
```
namespace AvePoint.GCommon.MicroKernel
{
    [ServiceContract(Namespace = "http://www.avepoint.com")]
    public interface ICoreService
    {
        [OperationContract]
        CoreMessage HandleMessage(CoreMessage message);
        [OperationContract]
        Boolean IsServiceRunning();
    }
}
```

接口的定义很简单，IsServiceRunning 用于测试服务是否可用，HandleMessage 用于处理消息，并且返回处理过的消息，CoreMessage 类型用于封装整个 DocAve 系统的消息调用，从当前的微核实现上讲，CoreMessage 类只支持一级继承，也就是说 CoreMessage 类的子类继承不受支持。其定义如下：

```
namespace AvePoint.GCommon.MicroKernel
{
    [DataContract(Namespace = "http://www.avepoint.com")]
    public class CoreMessage
    {
        [DataMember]
        public String Extension { get; set; }
        [DataMember]
        public CoreServiceInvocationContext InvocationContext { get; set; }
    }
}
```

其中 Extension 字段可以承载一些附加的信息，InvocationContext 字段是对调用上下文的封装，一般用于微核内部使用。如果为了拓展微核，可以直接在 CoreMessage 上标记 KnownType 特性，用于标识 CoreMessage 的子类型，如果没有源代码的情况下，请使用应用程序配置文件进行 KnownType 声明。

微核中对此定义进行了实现，具体实现位于 GCommon.MicroKernel 工程的 MicroKernelCoreService 文件夹下，如下：



在微核以前的版本的实现中，存在两个实现，一个是 `CoreService`，另一个是 `SingletonCoreService`，之所以这样做的原因是为了为不同的 `ServiceHost` 提供便利，在 IIS 中 Host 时，由于 IIS 中的 `Service` 的 `ServiceInstanceProvider` 依赖于 IOC 容器，并由 IOC 容器来管理实例的生命周期，在实例级别上没有办法声明相关的实例与并发管理属性，而为了在自寄宿中能够保持简洁的编程风格，因此在此采用了一个简单继承的编程结构。

在当前版本中，只有一个 `CoreService` 实现，之所以去掉单例实现，是因为 WCF4.0 之前单例模式下存在一个内存泄露的 bug，相关信息请参见以下链接：

<http://social.msdn.microsoft.com/Forums/en-US/wcf/thread/50e654ff-c4ab-4ccd-a855-ce90b95b1c13/>

<https://connect.microsoft.com/wcf/feedback/details/428811/memory-leak-in-wcf-singleton-services>

http://developers.de/blogs/damir_dobric/archive/2009/03/31/creating-of-proxies-inside-of-singleton-service.aspx

因此，微核中不再使用单例方式实现的 WCF 服务，同时在微核的承载一节中也对相应的服务承载方式做了变更。

`CoreService` 实现如下：

```
namespace AvePoint.GCommon.MicroKernel
{
    [AspNetCompatibilityRequirements(RequirementsMode =
                                    AspNetCompatibilityRequirementsMode.Allowed)]
    [ServiceBehavior(ConcurrencyMode = ConcurrencyMode.Multiple,
                    UseSynchronizationContext = false)]
    public class CoreService : ICoreService
    {
        public ICoreDispatcher Diapatcher { get; set; }
        #region ICoreService Members
        public CoreMessage HandleMessage(CoreMessage message)
        {
            var result = this.Diapatcher.DispatchMessage(message);
            return result;
        }
        public Boolean IsServiceRunning()
        {
            return !String.IsNullOrEmpty("I'm OK");
        }
        #endregion
    }
}
```

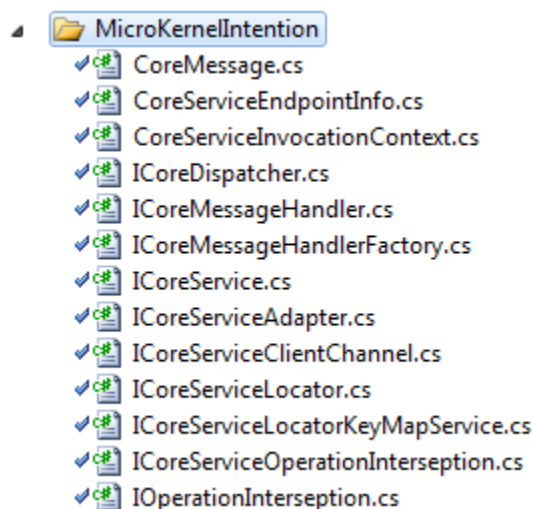
为了能聚焦问题的本质，上面的实现中已经精简过了，在 `HandleMessage` 的实现中，我们现在只关注 `Dispatcher` 对象，在 `Client-Dispatcher-Server` 模式的说明中，我们已经阐明了 `Dispatcher` 的作用，唯一没有说明的，就是静态的 `Dispatcher` 的实现方式，在微核的当前实现中，微核依赖于 IOC 容器，这点从实现上来讲，是为了在保持简洁编程风格的同时，提供弹性。

前面我们在论述微核的一般理论时提过，微核的实现是和 `Bridge` 模式（桥接模式）结合在一起的，请查看上述代码中的 `ICoreDispatcher` 接口，分发器在此处便是桥接模式的一个实现，从模式的角度来说，虽然 `Client-Dispatcher-Server` 模式是一个较为宏观的编程构造，但是桥接模式却关注的是微观的编程构造，从这个意义上说，这也是一种模式的模式，即组合模式（由

一些模式组合成另外一些较大模式) 的一个现实应用。

3.3.2. 微核功能意图定义

微核的功能意图定义在 `MicroKernelIntention` 文件夹, 全部基于 .Net 接口定义, 如下图所示:



在分发器的实现中, 我们应该重点关注以下接口:

`ICoreDispatcher` , `ICoreMessageHandler` , `ICoreMessageHandlerFactory` , `ICoreServiceLocator`

`ICoreDispatcher` 接口定义如下, 他的唯一作用就是分发消息:

```
public interface ICoreDispatcher
{
    CoreMessage DispatchMessage(CoreMessage message);
}
```

上文曾经指出, `Dispatcher` 的实现有多种方式, 微核中的实现如下:

```
public class CoreDispatcher : ICoreDispatcher
{
    public ICoreMessageHandlerFactory CoreMessageHandlerFactory { get; set; }
    public CoreMessage DispatchMessage(CoreMessage message)
    {
        var messageHandler = this.CoreMessageHandlerFactory.CreateMessageHandler(message.GetType().FullName);
        return messageHandler.HandlerMessage(message);
    }
}
```

(注: `CoreDispatcher` 类位于 `MicroKernelIntentionImpl` 文件夹下, 见名思意, 此文件夹中的类是对微核意图的实现)

诚如上文所言, 微核中的实现是配置文件加工厂委托, 分发器在内部引用了消息处理器工

厂类，委托工厂对相关消息进行进一步的处理。

ICoreMessageHandlerFactory 接口的作用，定义一个消息处理工厂，通过消息类型获得相应的消息处理程序，定义如下：

```
public interface ICoreMessageHandlerFactory
{
    ICoreMessageHandler CreateMessageHandler(String messageHandlerKey);
}
```

在获得相应的消息处理程序以后，由消息处理程序做进一步处理。在 CoreDispatcher 类的实现中，对 ICoreMessageHandlerFactory 仍然采取了桥接的实现方式，在微核的实现中，也确实存在两个不同的实现：DefaultCoreMessageHandlerFactory 和 DefaultIOCoreMessageHandlerFactory，在具体的实现中，他们不是直接实现接口而是由一个共同的抽象类来实现接口，并且完成线程同步以及对象缓存，这个类是 AbstractCoreMessageHandlerFactory，通过在这个抽象类，实现了模板方法模式，最终通过提供抽象方法

```
public abstract ICoreMessageHandler GetMessageHandler(String messageHandlerKey);
```

来与子类交互。

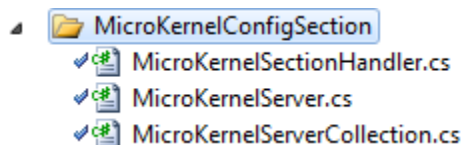
DefaultCoreMessageHandlerFactory 类实现了从应用程序配置文件中读取类型的能力，微核通过拓展应用程序配置文件的 SectionHandler 框架，实现了 MicroKernelSectionHandler，他使得微核能够处理自定义的 Section，其配置文件类型如下：

```
<microKernelSection>
  <MicroKernelServers>
    <MicroKernelServer id="Id" name="typeName" type="" path="assemblyPath" assemblyQualifiedType="" isSameAssembly="true" />
  </MicroKernelServers>
</microKernelSection>
```

为了集成 MicroKernelSectionHandler 的功能，需要在配置文件的开始位置添加以下的 sectionHandler

```
<configSections>
  <section name="microKernelSection" type="AvePoint.GCommon.MicroKernel.MicroKernelSectionHandler, CommonMicroKernel"/>
</configSections>
```

此处理程序的实现位于 MicroKernelConfigSection 文件夹下



其中上图中的类和配置文件中的节点是一一对应关系。核心实现如下：

```
var microKernelConfiguration = ConfigurationManager.OpenMappedExeConfiguration(
    new ExeConfigurationFileMap { ExeConfigFilename = "MicroKernelSection.config" }, Configur
```

```

ationUserLevel.None);
    microKernelSectionHandler = microKernelConfiguration.GetSection("microKernelSection") as MicroKernelSectionHandler;
    result = microKernelSectionHandler.MicroKernelServers[messageHandlerKey].AssemblyQualifiedType;

```

获得类型的程序集全名以后,便可以得到类型。其中类型实现了 `ICoreMessageHandler` 接口。

`DefaultIOCoreMessageHandlerFactory` 内部使用了 `IServiceLocator` 接口来发现类型,实现如下:

```

public class DefaultIOCoreMessageHandlerFactory:AbstractCoreMessageHandlerFactory
{
    public ICoreServiceLocator IServiceLocator { get; set; }
    public override ICoreMessageHandler GetMessageHandler(String messageHandlerKey)
    {
        return this.ServiceLocator.Discover<ICoreMessageHandler>(messageHandlerKey);
    }
}

```

在微核的使用中,为了结合 IOC 容器的强大功能,默认使用 `DefaultIOCoreMessageHandlerFactory` 类,在微核中,为了适应这种情形,提供了 `DefaultIOServiceLocator` 类,服务定位发现的过程被实现为从 IOC 容器中取得,同时提供 `ICoreServiceLocatorKeyMapService` 接口,用于对默认的发现规则进行局部调整。

在 `DefaultIOServiceLocator` 类的实现过程中,也使用了模板方法模式,以获得设计上的便利以及基础设施的重用。`DefaultIOServiceLocator` 不是直接实现 `ICoreServiceLocator` 接口,而是拓展自顶层的抽象类。

3.3.3. 实现外部服务器

根据服务定位器的功能,特别是 `DefaultIOServiceLocator` 的功能,从 IOC 容器中发现了消息类型特定的消息的处理程序,在微核中,内置了一个最核心的 `CoreMessage` 类型消息的处理程序:

`DefaultCoreMessageHandler`,与 `MessageHandlerFactory` 的设计类似,`CoreMessageHandler` 的处理层次上也使用了模板方法,但与之不同的是,为了支持编译时的强类型编程,在抽象类的部分采用了泛型抽象基类,并在消息中实现了标准的 `Dispose` 模式。`CoreMessageHandler` 的声明如下:

```

public abstract class CoreMessageHandler<Message> : ICoreMessageHandler where Message : CoreMessage,
    new()
{
    public abstract Message ProcessMessage(Message message);
    public CoreMessage HandlerMessage(CoreMessage coreMessage)
    {
        var result = default(CoreMessage);
        var processMethod = this.GetType().GetMethod("ProcessMessage");
        var genericArgType = this.GetType().BaseType.GetGenericArguments()[0];

```

```
        result = processMethod.Invoke(this, new Object[] { coreMessage }) as CoreMessage;
        return result;
    }
}
```

在子类的层次上，通过覆盖模板方法，进行消息处理，DefaultCoreMessageHandler 的声明如下：

```
public class DefaultCoreMessageHandler : CoreMessageHandler<CoreMessage>
{
    public override CoreMessage ProcessMessage(CoreMessage message){ }
}
```

如果通过这种方式来实现另外的一个外部服务器，可以通过实现具体的子类，把相应的子类注入容器即可从微核中截获消息，例如 Media 模块想要从微核中加入消息处理流程，采取以下步骤，实现一个 CoreMessageHandler<T>的子类，如下：

```
public class MediaCoreMessageHandler : CoreMessageHandler<MediaMessage>
{
    public override MediaMessage ProcessMessage(MediaMessage message)
    {
        return message;
    }
}
```

将此类向 IOC 容器注册：

```
<component id="AvePoint.GCommon.Contract.Media.Object.MediaMessage" service="AvePoint.GCommon.MicroKernel.ICoreMessageHandler, CommonMicroKernel" type="AvePoint.Media.Service.Rpc.InWay.MediaCoreMessageHandler, MediaServiceCommon" />
```

则，MediaCoreMessageHandler 就会加入到处理消息的 pipe 中。

从整个流程看，MediaCoreMessageHandler 是作为一个外部服务器存在的，在微核中的信息流，最终会通过某种方式流向外部服务器进行最终处理。

至此，微核的基本任务都已经实现了，当从整体上看，以上的这种外部服务器拓展 CoreMessage 的结构是有限制的，提供的灵活性不够，因为他必须指定 Message 的类型，甚至于在没有提供客户端代理或者适配器的情况下，不得不自己处理与微核的交互，为了使微核的核心功能摆脱这些限制，基于 DefaultCoreMessageHandler 类，也就是基于 CoreMessage 消息对调用进行封装。从这种意义上来说，外部服务器的概念被大大的拓展了。

通过对基于栈的方法调用转化为基于消息的远程调用，外部服务器已经没有什么实质的限制了，因为任意的调用都被转化为消息，唯一需要做的就是 Service 的发现过程，而发现外部服务器的过程又结合了 IOC 的便利功能，因此基于消息的拓展的主要作用在微核中主要表现为对以前模块的兼容，主要原因是因为一些模块先于微核模块开发，如 CA 等。

3.3.4. 适配器/代理以及核心内部服务器

3.3.4.4. 适配器/代理的实现

作为与微核交互的核心应用程序编程接口，适配器代理机制被定义为一个核心的接口：

`IProxyBuilder` 接口，定义如下：

```
public interface IProxyBuilder
{
    TInterface CreateProxy<TInterface>(CoreServiceEndpointInfo endpoint);
    Object CreateProxy(Type interfaceType, CoreServiceEndpointInfo endpoint);
    Object CreateProxy(String interfaceTypeAssemblyQualifiedName, CoreServiceEndpointInfo endpoint);

    event EventHandler<ProxyEventArgs> PreProxyInvoke;
    event EventHandler<ProxyEventArgs> PostProxyInvoke;
}
```

在此接口的当前版本实现中，提供基础功能的是泛型 `CreateProxy` 方法，此方法的主要作用是创建一个动态创建一个接口的实例，此接口的实例将对微核发起调用，但对此实例的调用者来说，他不需要也不应该知道服务如何定位与发现，如何与服务进行交互，他只关心是否有一个能满足自己需要的服务就可以了。`CreateProxy` 其余的两个重载是特定情况下为了调用方便使用的，比如说后期绑定的情况下，没有办法提供强类型的接口类型，此时重载就有了用武之地。它利用运行时类型发现，可以动态的调用适配器/代理。

此外，接口中还提供了两个事件 `PreProxyInvoke` 和 `PostProxyInvoke`，这两个事件的主要意图是，提供与适配器底层进行交互的能力，从一般意义上讲，适配器不需要对底层的运行时施加影响，但是为了搭建一个开放式的结构，这两个事件公开了对底层实现的部分细节，可以在适配器或代理对微核发起调用之前和之后，进行拦截操作或者对底层的消息施加影响等。由于当前的底层交互使用的是 `Wcf` 技术，所以这种影响主要是对 `Wcf` 客户端运行时的影响。为了配合使用这种功能，需要相应的拓展微核，而这种拓展，将进入微核的 `PipeFilter` 处理管道（详见下文）。

```
public TInterface CreateProxy<TInterface>(CoreServiceEndpointInfo coreServiceEndpoint)
{
    var endpoint = new EndpointInfo
    {
        EndpointConfigurationName = coreServiceEndpoint.EndpointConfigurationName,
        HostOrIpAddress = coreServiceEndpoint.HostOrIpAddress,
        Port = coreServiceEndpoint.Port,
        Scheme = endpoint.Scheme,
        RemotingTypeKey = endpoint.RemotingTypeKey
    };
    var remoteProxy = new RemotingProxy<TInterface>(endpoint);
    remoteProxy.PreProxyInvoke += PreProxyInvoke;
    remoteProxy.PostProxyInvoke += PostProxyInvoke;
    return (TInterface)remoteProxy.GetTransparentProxy();
}
```

在 CreatePorxy 方法的实现中，采用了 .Net Remoting 远程处理层的技术，通过使用远程处理层的相应 API，实现了不依赖于第三方组件的方法来完成[栈调用向消息调用的转换](#)，在此版本的实现中，需要明确两个技术概念：真实代理 RealProxy 与透明代理 TransparencyProxy。

- 真实代理 RealProxy，它提供基本的代理功能，RealProxy 类是代理必须继承的 abstract 基类。客户端在跨任何类型的远程处理边界使用对象时，对对象使用的实际上是透明代理。透明代理使人以为实际对象驻留在客户端空间中。它实现这一点的方法是：使用远程处理基础结构将对其进行的调用转发给真实对象。
- 透明代理 TransparencyProxy，它本身由 RealProxy 类型的托管运行时类的实例收容。RealProxy 实现从透明代理转发操作所需的部分功能。请注意，代理对象继承与托管对象（如垃圾回收、字段和方法支持）关联的语义，并且可以扩展以形成新类。该代理具有双重特性：它作为与远程对象（透明代理）相同的类的对象，同时其本身是托管对象。

最终，调用将被转发到微核的内部泛型类 RemotingProxy<TInterface>上，此类实现了远程处理基础结构所要求的条件。其核心方法如下：

```
public override IMessage Invoke(IMessage msg)
{
    var context = new InvocationContext { Request = (IMethodCallMessage)msg };
    PrivateInvoke(endpoint, context);
    return context.Reply;
}
```

在 Invoke 内部将由 PrivateInvoke 私有方法对微核发起调用。此调用通过底层的 Wcf 来完成。

3.3.4.4. 核心内部服务器

核心内部服务器是由类 DefaultCoreMessageHandler 类实现的，此类依托于 CoreMessage 类对原始的调用上下文的封装，封装由 CoreServiceInvocationContext 类完成，定义如下：

```
public class CoreServiceInvocationContext
{
    public String TypeKey { get; set; }
    public String TypeName { get; set; }
    public String MethodName { get; set; }
    public Int32 ArgsCount { get; set; }
    public List<String> Args { get; set; }
    public List<String> ArgsTypeNames { get; set; }
    public List<String> GenericParameterTypeNames { get; set; }
    public String ReturnValue { get; set; }
    public String Uri { get; set; }
}
```


通过此类，完成了以下功能：

首先在客户代码中完成由栈调用转化为消息调用，然后将远程处理基础结构提供的消息进一步包装为 `CoreServiceInvocationContext`，然后通过底层 Wcf 机制对核心的内部服务器发起调用。在核心内部服务器内部，将对 `CoreServiceInvocationContext` 消息进行解析，通过服务定位机制，查找到外部服务器，然后由核心服务器在内部将消息调用转化为栈调用，此时，基于客户端的适配器或者代理的调用，最终转发给相应的外部服务器。

外部服务器在完成调用以后，需要完成调用上下文的传递，此上下文会通过 `CoreServiceInvocationContext` 类进行封装，进行回传，在上下文返回以后，微核内部的 `RemotingProxy` 类会对 `CoreServiceInvocationContext` 封装的上下文进行解析，然后对解析的内容进一步转换为远程处理基础结构的消息调用上下文，包括包装异常信息，释放底层信道等。

`DefaultCoreMessageHandler` 类的核心方法是 `ProcessMessage`，实现如下：

```
public override CoreMessage ProcessMessage(CoreMessage message)
{
    var invocationContext = message.InvocationContext;
    var invocationInstance = this.ServiveLocator.Discover(invocationContext.TypeKey);
    var argsTypes = GetInvocationArgsTypes(invocationContext);
    var args = GetInvocationArgs(invocationContext, argsTypes);
    var methodInfo = invocationInstance.GetType().GetMethod(invocationContext.MethodName);
    if (methodInfo.IsGenericMethodDefinition)
    {
        var genericParameterTypes = GetInvocationGenericParameterArgs(invocationContext);
        methodInfo = methodInfo.MakeGenericMethod(genericParameterTypes);
    }
    var result = methodInfo.Invoke(invocationInstance, args);
    if (result != null)
        invocationContext.ReturnValue = SerializerHelper.SerializeToBase64StringByDataContractSerializer(result);
    invocationContext.ReturnValueTrueType = result.GetType().AssemblyQualifiedName;
    return message;
}
```

上文提到，微核的代码是最丑陋的代码之一，从这个核心内部服务器的实现上，我们可以看到代码是比较丑陋和难以理解的，上面方法的主要作用是通过各种方式查找实例，调用方法，参数，发起调用，封装返回值等操作。实现过程尽可能的使用 C# 的简洁语法，以压缩代码量。

3.3.4.4. 微核内部终结点缓存策略

微核底层使用使用 WCF 通讯技术，为了维持底层的 WCF 的连接的安全可靠的连接，微核内置了一个终结点缓存策略，通过在底层维持一个持续可用的连接，使得微核的使用者不需要关心也不会意识到底层对连接的保持工作，同时通过对整个消息调用链的保持，使得微核的使用者感

觉整个调用过程本身和调用一个本地方法直观上没有什么明显区别。

微核的终结点缓存策略如下：

- 终结点缓存的最大数量为 50
- 微核始终保持在最大数量范围内的终结点是可用的，除非服务端当机，也就是说，微核客户端不会造成连接失效
- 为了保持超时检查，微核每 5 分钟对核心服务检查一次，如果在 5 分钟之内没有调用的话。

缓存策略代码在 `ClientChannelManager` 类中实现，在实现缓存策略的同时，也实现了并发处理，线程同步等措施。

3.3.5 微核的附加功能

为了能够和适配器代理提供对称的功能增强，在微核使用的信道一级，我们提供了基于 Pipe/Filter 模式的消息拦截功能，以便能针对代理对附加的信息进行进一步过滤或者处理。同时为了能够在运行时进一步的确认问题，微核拓展了 .net 的跟踪系统，实现了自己的跟踪框架。

3.3.5.1 微核的 Pipes/Filter 模式拦截器

前文说过，Pipes/Filter 模式类似一个顺序工作流，在流程中，我们可以顺序的插入拦截器，从本质上说，这些拦截器就是 Filter，微核中，拦截器被定义为一个接口：

```
public interface IOperationInterseption
{
    void PreInvoke(InterseptionContext context);
    void PostInvoke(InterseptionContext context);
}
```

在配置了拦截器的系统中，可以通过实现实现这两个接口方法实现对微核底层的消息进行拦截，注意此拦截器的实现基于 Wcf 通讯层之上，是微核自己领域的规则。

所有配置的拦截器都会在微核中整合成 `CoreService` 的一个拦截器，此功能的通过在 `CoreService` 的实现中内置一个 `ICoreServiceOperationInterseption` 对象实现的

```
public interface ICoreServiceOperationInterseption
{
    void PreCoreServiceOperationInvoke(InterseptionContext context);
    void PostCoreServiceOperationInvoke(InterseptionContext context);
}
```

基于当前 DocAve 系统的开发进度，在实际应用中并没有配置任何的拦截器，此模块只供将来拓展使用。

3.3.5.2 微核跟踪系统

为了在运行时能够有效的发现问题，微核实现了一个 .net 标准的基于源的跟踪系统，此系统由接口 `ITraceSource` 定义，并提供了可拓展功能（下文将介绍 `Media` 模块对此跟踪系统的一个拓展），跟踪与 `log` 类似，但在 `log` 之上提供了更灵活的控制，它使得微核具有了一种只针对微核一个逻辑结构输出信息的能力，这个是 `log` 系统所不具备的。

微核中定义的跟踪源名称与程序集同名，定义在 `MicroKernelConstant` 类中的静态只读字段 `MicroKernelTraceSource` 代表了跟踪源的名称，通过在微核放置跟踪语句，就可以实现对微核跟踪输出的单独控制。

结合 .net 应用程序配置文件的设置，可以实现复杂的跟踪效果，一个成熟配置文件的例子如下：

```
<?xml version="1.0" encoding="utf-8" ?>
<system.diagnostics>
  <sources>
    <source name="CommonMircoKernel" switchType="System.Diagnostics.SourceSwitch"
switchName="CommonMicroKernelSwitch">
      <listeners>
        <add name="console"/>
      </listeners>
    </source>
  </sources>
  <sharedListeners>
    <add name="console" traceOutputOptions="ProcessId, ThreadId, Timestamp, DateTime" type="
System.Diagnostics.ConsoleTraceListener" initializeData="false" />
  </sharedListeners>
  <switches>
    <add name="CommonMicroKernelSwitch" value="Error"/>
  </switches>
  <trace autoflush="true" indentsize="4" >
    <listeners>
      <add name="console" />
    </listeners>
  </trace>
</system.diagnostics>
```

此配置来源于一个成熟的模块，它将把微核的跟踪消息输出到控制台，一旦出现问题时，可以结合微核底层 `Wcf` 跟踪消息与微核层面的跟踪消息，来快速准确的定位问题。

4 微核应用

本节，我们将就如何使用微核框架进行阐述，下面的例子我们将基于 Manager/Media 模块来进行展开，因为这两个模块之间最先使用微核框架实现相关功能。

4.1 微核的承载

使用微核的第一步是如何使微核的启动工作，本质上，微核的承载工作等同于一个 Wcf 服务的承载过程，但在 Manager 和 Media 部门却使用了不同的方式。

4.1.1 DocAve6 Manager 中的微核承载

在 Manager 中，借助于 Manager 业已提供的框架，微核通过 IIS SVC 文件的方式公开为一个 Wcf 服务，此 SVC 服务的物理文件位于 Server\VcManager\VCWeb\ControlCore\ControlCoreService.svc，由于 Manager 端的 Wcf 实例提供模型依赖于 Castle Windsor 容器，因此 Wcf 的实例管理与生命周期管理由 Wcf 框架转移给了 Castle 框架，同时，微核借助于在 Manager 的 Web.config 中引入微核的驱动配置文件 CommonMicroKernellocConfigurations.config 来实现了微核的正常运行。一般说来，微核在 IIS 中的运行和一般的 Wcf 服务没有任何区别，只是需要单独的引入驱动微核的配置文件。

此 SVC 文件的内容如下图所示：

```
<%@ ServiceHost Language="C#" Debug="true" Service="AvePoint.GCommon.MicroKernel.CoreService"%>
```

4.1.2 DocAve6 Agent, Media, Report 服务中的微核承载

不同于 IIS 的承载方式，微核核心服务的承载方式采用了自承载的方式，如前文指出的，微核不再使用单例的承载方式来结合 WCF 以及 IOC 框架，同时微核也从以前的一个可启动的组件退化为一个普通的组件，为了维持之前代码的一致性，仍然实现为一个 Start 方法，同时也为了结合 IOC 以及 WCF 框架，自定义服务宿主：WindsorServiceHost，代码如下：

```
public void Start()
{
    this.serviceHost = new WindsorServiceHost(this.windsorContainer, typeof(CoreService),
        (new List<Uri> { new UriBuilder().Uri }).ToArray());
    serviceHost.Open();
}

internal class WindsorServiceHost : ServiceHost
{
```

```

I WindsorContainer windsorContainer;
public WindsorServiceHost(I WindsorContainer container, Type serviceType, Uri[] baseAddresses)
    : base(serviceType, baseAddresses)
{
    this.windsorContainer = container;
}
public WindsorServiceHost(I WindsorContainer container, Type serviceType)
    : this(container, serviceType, null) { }
protected override void OnOpening()
{
    base.OnOpening();
    if (this.Description.Behaviors.Find<WindsorDependencyInjectionServiceBehavior>() == null)
        this.Description.Behaviors.Add(new
            WindsorDependencyInjectionServiceBehavior(this.windsorContainer));
}
}

```

为了把 WCF 的实例创建过程委托给 IOC 容器，我们拓展了 WCF 的框架的实例提供接口，通过在 IOC 容器中获取 WCF 服务对象的实例，使得每一个 WCF 请求，都会从容器中获得一个请求的响应对象。通过这种机制，微核运转起来。最后你还需要在相关的服务中引入微核的配置。如下如所示：

```

<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <configSections>
    <section name="castle" type="Castle.Windsor.Configuration.AppDomain.CastleSectionHandler, Castle.Windsor" />
  </configSections>
  <system.serviceModel>
    <services configSource="MediaWcfServicesConfigurations.config" />
    <client configSource="MediaWcfClientConfigurations.config" />
    <behaviors configSource="MediaWcfBehaviorsConfigurations.config" />
    <bindings configSource="MediaWcfBindingsConfigurations.config" />
  </system.serviceModel>
  <system.diagnostics configSource="MediaTraceConfigurations.config" />
  <castle>
    <include uri="file://CommonMicroKernelIocConfigurations.config" />
    <include uri="file://MediaIocPropertiesConfigurations.config" />
    <include uri="file://MediaIocConfigurations.config" />
    <include uri="file://MediaIocFacilitiesConfigurations.config" />
  </castle>
</configuration>

```

其中微核相关的终结点信息都是从依托容器的环境变量中取得。

4.2 使用微核

前文阐述，适配器/代理就是微核的 API，这在微核的定义中就是 IProxyBuilder 接口，并在微核中内置了实现类 RemotingProxyBuilder，基本上调用代码会编程下面的样子，此实例取自 Media 中的一段测试用例，如下图所示：

```

var builder = new RemotingProxyBuilder();
var mediaService = builder.CreateProxy<ITestMedia>(new CoreServiceEndpointInfo

```

```
{  
    EndpointConfigurationName = "mediaAgentCoreService",  
    HostOrIpAddress = "10.2.6.18",  
    Port = 12001,  
    Scheme = "net.tcp"  
});  
  
var result = mediaService.Response("Hi, Lance");
```

此段示例代码对位于 10.2.6.18 的主机的 12001 端口发起了调用,调用接口为 ITestMedia,方法为 Response,在实现这个效果的过程中,只有一个 Service 和 Client 的 EndPoint 参与其中,即使以后增加新的调用,也不会再增加服务和终结点,也就是说,配置文件做好初始配置以后,便是最终的结果了。

Media 中配置文件的 Service 节点和 EndPoint 配置如下:

```
<service name="CoreServiceConfiguration" behaviorConfiguration="globalDefaultServiceBehavior">  
    <endpoint name="mediaMainCoreService" address="mediaAgentCoreService" binding="customBinding"  
bindingConfiguration="netTcpWithServerClientSecurityCustomBinding" contract="AvePoint.GCommon.Contract.ICoreService" />  
</service>  
<client>  
    <endpoint address="https://0.0.0.0:0/ControlCore/ControlCoreService.svc"  
binding="basicHttpBinding"  
bindingConfiguration="globalDefaultBasicHttpBinding"  
contract="AvePoint.GCommon.Contract.ICoreService"  
name="managerControlService" />  
</client>
```

由于 Media 只对 ControlService 发起调用,因此只需要配置指向 Control 的终结点即可,在真实的对 Control 发起调用的过程中,上述测试代码的 Scheme 需要为 https,EndpointConfigurationName 需要为对应的 managerControlService 的终结点名称。

在 Media 中,借助 Castle Windsor 容器,对固定终结点的地址,端口进行进一步的包装,封装了静态的 MediaProxyBuilder 类:

```
public static class MediaProxyBuilder  
{  
    public static TInterface CreateProxy<TInterface>()  
    {  
        var innerProxyBuilder = MediaEnvironment.ProxyBuilder;  
        var innerEndPointInfo = MediaEnvironment.EndPointInfo;  
        return innerProxyBuilder.CreateProxy<TInterface>(innerEndPointInfo);  
    }  
}
```

通过封装此类，调用关系进一步简化，以测试用例为例，将变为以下结果：

```
var mediaService = MediaProxyBuilder.CreateProxy<ITestMedia>();  
var result = mediaService.Response("Hi, Lance");
```

通过这种形式，我们只需要进行实现一个 ITestMedia 接口，在被调用方将接口以及实现作为 IOC 即 Castle Windsor 容器的组件注册即可

```
<component id="AvePoint.GCommon.Contract.Media.ITestMedia" service="AvePoint.GCommon.Contract.Media.ITestMedia, CommonContract" type="AvePoint.GCommon.Contract.Media.TestMediaClass, CommonContract" />
```

此过程中，我们没有配置任何关于服务和终结点的信息，只是添加了一个组件到容器。注意此处的 id 需要是接口的全名，或者在容器节点中对接口全名和 id 进行了映射，如下面示例代码中注释掉的部分演示的那样。

```
<component id="AvePoint.GCommon.MicroKernel.ICoreServiceLocatorKeyMapService" service="AvePoint.GCommon.MicroKernel.ICoreServiceLocatorKeyMapService, CommonMicroKernel" type="AvePoint.GCommon.MicroKernel.CoreServiceLocatorKeyMapService, CommonMicroKernel">  
  <parameters>  
    <KeyMapDictionary>  
      <dictionary>  
        <!--entry key="AvePoint.GCommon.Contract.Media.ITestMedia">AvePoint.GCommon.Contract.Media.ITestMedia</entry-->  
      </dictionary>  
    </KeyMapDictionary>  
  </parameters>  
</component>
```

特别针对 Media 的模块，需要阐明的是，调用 Media 需要配置以下的终结点，此终结点为一个固定的配置：

```
<endpoint address="net.tcp://0.0.0.0:0/mediaAgentCoreService"  
  binding="customBinding"  
  bindingConfiguration="netTcpWithServerClientSecurityCustomBinding"  
  behaviorConfiguration="globalDefaultEndPointBehavior"  
  contract="AvePoint.GCommon.Contract.ICoreService"  
  name="mediaAgentCoreService" />
```

最后，也是使得微核功能正常工作的最重要的一行代码是：

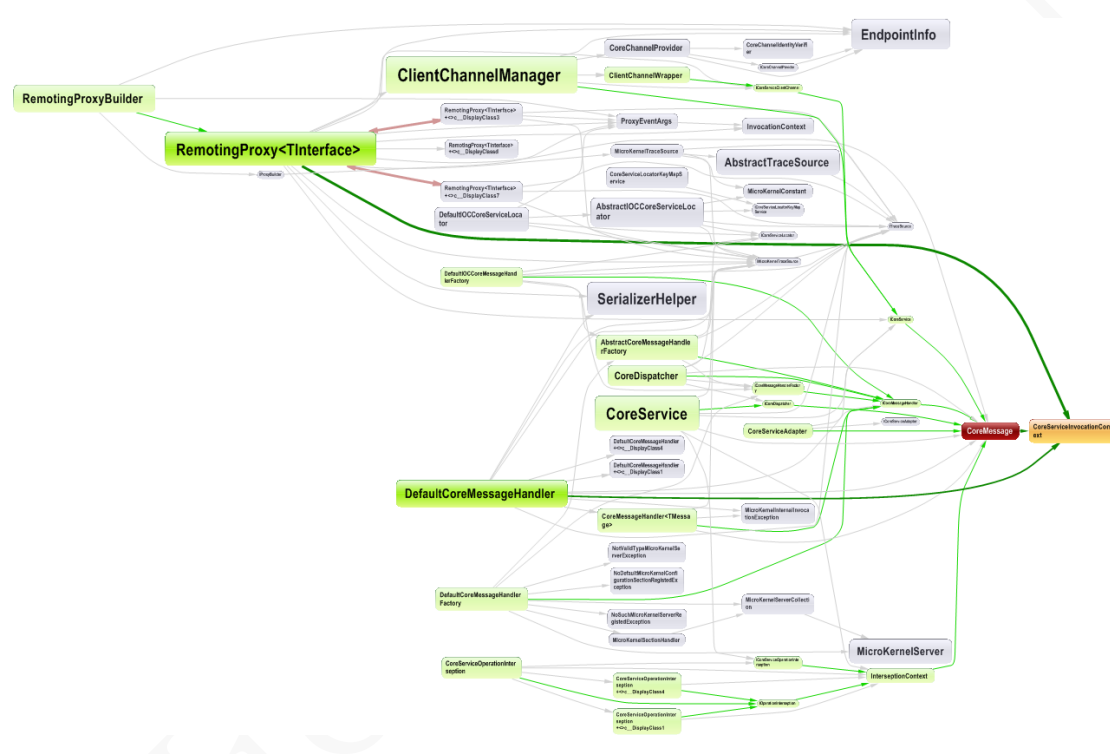
```
AppDomain.CurrentDomain.SetData(ContractConstants.CORE_IOC_CONTAINER_IDENTIFIER, windsorContainer);
```

这行代码的主要作用是，将微核进行应用程序域级别的注册，以配合微核内部的服务定位器正常工作。服务定位器从应用程序域中查找容器，并进一步从容器中定位对象。这行代码需要在启动容器以后立即执行。

MicroKernel

5 关于微核的最后思考

MicroKernel 架构是一个很大的模式，在实现时就像是外部服务器的容器一样，这也是实现微核时所采用的方法，但是在 DocAve6 中我们绝不是想将微核转化为一个解决复杂问题的容器，微核模块仅仅是委托功能，而不是自己实现功能。整个微核的功能可以使用下午来描述，此图使用了微核中使用的真实类型来反映通过微核的消息流：



MicroKernel 模式对组成一个完整微核的各个部分都进行了明确的区分和定义，并且只实现了每部分必须的责任。适配器将微核和内部服务器连接在一起并实现业务逻辑，微核保持聚焦性并将重点放在了应用程序的整个结构上，内部服务器连接平台细节和微核，外部服务器处理大量业务逻辑。通过单一责任贯彻以及有机配合的设计，应用程序实现了功能的分离与业务逻辑的松散耦合，并使得微核的未来拓展更加简单。由此我们发展起来一个足以迎接未来需求变更的微核系统。

总之，通过功能委托以及对服务概念的更高层次抽象与整合，DocAve 微核系统不仅使管理日益膨胀的服务的目标由可能成为现实，还把 DocAve 整合成一个有机的 SOA 的可拓展系统。

MicroKernel